Assignment 3

Database Design Analysis

Redundancy, anomalies, functional dependencies, and normalization

Project: LibSphere - Library Management System **Prepared:** 23 October 2026

Platform assumption: MySQL 8.0.16 or later (InnoDB). Outputs in this report are expected results derived from the included sample dataset; run the SQL in MySQL Workbench to capture authentic execution screenshots if required by the instructor.

The source package describes LibSphere and also includes earlier ER/relational sketches. The older sketches mix a free-text Author attribute in Books with separate publisher/member records, which can repeat facts and make relationships harder to enforce. For the remaining assignments, this report standardizes on the six-table LibSphere specification.

1. Potential redundancy in the earlier design

Design issue	Why it is a problem	Improvement
Author stored as a text value inside Books	Co-authors need lists or repeated text, spelling differences are easy, and author-level searching is unreliable.	Store each author once in Authors and connect books through Book_Authors.
Publisher name/address repeated in every book row in the older sketch	Changing a publisher address requires updating many rows; missed updates create contradictory values.	If publisher tracking is required, create a Publishers table and reference publisher_id from Books. It is not part of the finalized six-table scope.
Member details copied into borrowing/issue records	A phone or email change must be reflected in all related rows; historical rows may disagree.	Keep member identity/contact data in Members and store only member_id in Loans.
Category name repeated in book records	Repeated descriptive text causes update and spelling inconsistencies.	Store category_name once in Categories and reference category_id.
Available copies stored alongside total copies	available_copies can drift from total copies and open-loan counts if updates are not transactional.	Keep the count only if checkout/return logic updates it atomically; periodically reconcile it against outstanding loans, or derive it when appropriate.

2. Insertion, update, and deletion anomalies

Anomaly	Example in a poorly designed library table	Impact
Insertion	An author or category cannot be recorded until at least one book row exists, or a new author requires stuffing multiple names into one column.	The system cannot represent real information independently and may require placeholder values.
Update	The same publisher address or member email appears in many rows and only some copies are changed.	The database contains conflicting facts and reports may return inconsistent results.
Deletion	Deleting the only book row associated with an author/publisher also deletes the only stored copy of that author/publisher information.	A deletion for one purpose unintentionally removes unrelated information.

These anomalies are caused by storing several independent entities in one relation, duplicating attributes that describe the same real-world object, or using repeating groups instead of atomic values. Constraints can prevent some invalid values, but normalization is needed to remove the underlying structural causes.

3. Functional dependencies in the finalized schema

Relation	Main functional dependency	Notes
Categories	category_id -> category_name	category_name is also unique, so category_name -> category_id.
Authors	author_id -> first_name, last_name, bio	Author details are stored once per author.
Books	book_id -> title, isbn, category_id, publication_year, total_copies, available_copies	isbn -> book_id because ISBN is UNIQUE.
Book_Authors	(book_id, author_id) -> no non-key attributes	Composite primary key prevents duplicate links; each component references its parent.
Members	member_id -> first_name, last_name, email, phone, membership_date, status	email -> member_id because email is UNIQUE.
Loans	loan_id -> book_id, member_id, loan_date, due_date, return_date, status	Each loan_id identifies one borrowing transaction.

Notation: X -> Y means the values of X determine the values of Y. The primary key or declared unique candidate key determines the non-key attributes in each table. For Book_Authors, the key is the pair (book_id, author_id); neither column alone uniquely identifies every relationship row.

4. Normalization and proposed improvements

- **First Normal Form (1NF):** keep every cell atomic. Do not store a comma-separated list of authors in Books.
- **Second Normal Form (2NF):** in the Book_Authors bridge, keep only the relationship keys unless an attribute depends on the complete book-author pair. This avoids partial dependency on one part of a composite key.
- **Third Normal Form (3NF):** store category labels and author/member details in their own tables. A book stores category_id, not category_name; a loan stores member_id, not the member phone/email.
- **Integrity:** use primary keys, foreign keys, UNIQUE, CHECK, NOT NULL, and DEFAULT constraints. Use RESTRICT for deleting books or members that still have loan history; cascade only on the pure bridge rows.
- **Operational consistency:** perform issuing/returning and available-copy changes in a transaction, with concurrency controls. A trigger or application service may maintain stock; avoid updating the same stock count in multiple places independently.

5. Final relationship diagram

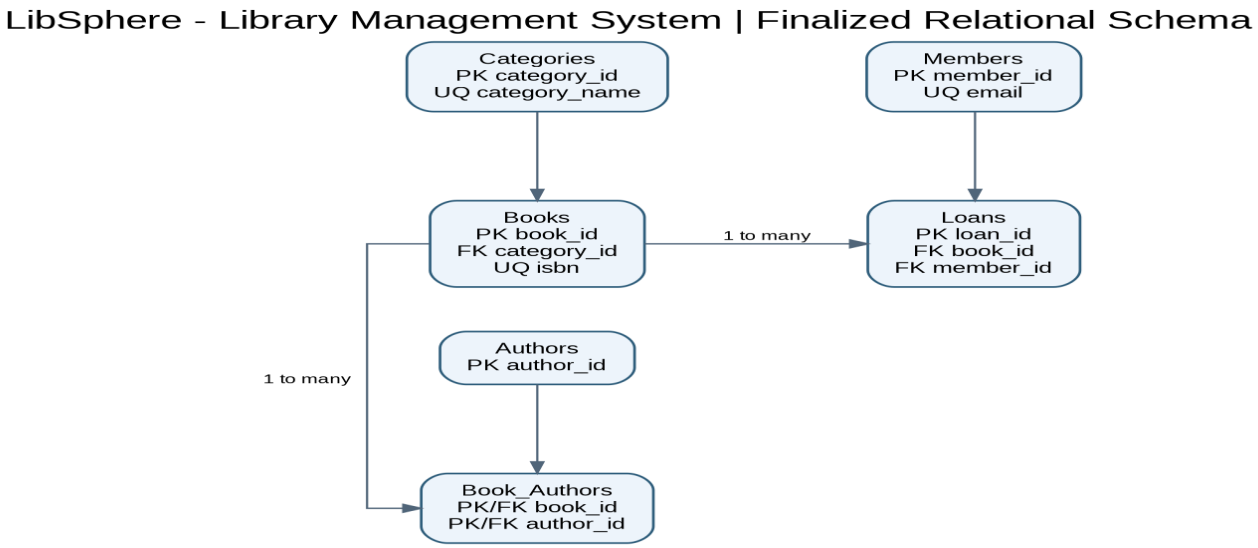


Figure 1. Finalized LibSphere relationships: a category has many books; books and authors connect through a bridge table; books and members connect to borrowing records through Loans.

Conclusion: the six-table design separates independent entities, resolves the many-to-many book-author relationship, and makes common insertion/update/deletion anomalies much less likely. Inventory counts and loan statuses still require consistent transaction logic because structural normalization alone does not synchronize operational values.